Overview of Deep Q-network

Subject: Deep Q-network

1.

Animal behavior detection CNNs have been applied in ecological and behavioral research to automatically detect and quantify animal behavior from visual data, enabling identification of animals, tracking of individuals, estimation of pose, and classification of specific actions such as feeding, and social interactions. Combined with multi-object tracking and temporal modeling, these systems can extract behavioral sequences over extended recordings, reducing reliance on manual annotation and increasing throughput for studies of individual variation, social networks, and collective dynamics.

2.

Parameter sharing A parameter sharing scheme is used in convolutional layers to control the number of free parameters. It relies on the assumption that if a patch feature is useful to compute at some spatial position, then it should also be useful to compute at other positions. Denoting a single 2-dimensional slice of depth as a depth slice, the neurons in each depth slice are constrained to use the same weights and bias. Since all neurons in a single depth slice share the same parameters, the forward pass in each depth slice of the convolutional layer can be computed as a convolution of the neuron's weights with the input volume. Therefore, it is common to refer to the sets of weights as a filter (or a kernel), which is convolved with the input. The result of this convolution is an activation map, and the set of activation maps for each different filter are stacked together along the depth dimension to produce the output volume. Parameter sharing contributes to the translation invariance of the CNN architecture. Sometimes, the parameter sharing assumption may not make sense. This is especially the case when the input images to a CNN have some specific centered structure; for which we expect completely different features to be learned on different spatial locations. One practical example is when the inputs are faces that have been centered in the image: we might expect different eye-specific or hair-specific features to be learned in different parts of the image. In that case it is common to relax the parameter sharing scheme, and instead simply call the layer a "locally connected layer". In this layer, the convolutional kernels' parameters are not shared. Instead, the network learns independent weights and biases for each spatial location. This allows each location to have its own feature-learning ability, making it better suited to handle images with distinct central structures or irregular features.

3.

Go CNNs have been used in computer Go. In December 2014, Clark and Storkey published a paper showing that a CNN trained by supervised learning from a database of human professional games could outperform GNU Go and win some games against Monte Carlo tree search Fuego 1.1 in a fraction of the time it took Fuego to play. Later it was announced that a large 12-layer convolutional neural network had correctly predicted the professional move in 55% of positions, equalling the accuracy of a 6 dan human player. When the trained convolutional network was used directly to play games of Go, without any search, it beat the traditional search program GNU Go in 97% of games, and matched the performance of the Monte Carlo tree search program Fuego simulating ten thousand playouts (about a million positions) per move. A couple of CNNs for choosing moves to try ("policy network") and evaluating positions ("value network") driving MCTS were used by AlphaGo, the first

to beat the best human player at the time.

4.

Fully connected layers Fully connected layers connect every neuron in one layer to every neuron in another layer. It is the same as a traditional multilayer perceptron neural network (MLP). Each neuron in the fully connected layer receives input from all the neurons in the previous layer. These inputs are weighted and summed with the corresponding biases, and then passed through an activation function to perform a nonlinear transformation, generating the output. The flattened matrix goes through a fully connected layer to classify the images.

5.

Shift-invariant neural network A shift-invariant neural network was proposed by Wei Zhang et al. for image character recognition in 1988. It is a modified Neocognitron by keeping only the convolutional interconnections between the image feature layers and the last fully connected layer. The model was trained with back-propagation. The training algorithm was further improved in 1991 to improve its generalization ability. The model architecture was modified by removing the last fully connected layer and applied for medical image segmentation (1991) and automatic detection of breast cancer in mammograms (1994). A different convolution-based design was proposed in 1988 for application to decomposition of one-dimensional electromyography convolved signals via de-convolution. This design was modified in 1989 to other de-convolution-based designs.

6.

A convolutional neural network consists of an input layer, hidden layers and an output layer. In a convolutional neural network, the hidden layers include one or more layers that perform convolutions. Typically this includes a layer that performs a dot product of the convolution kernel with the layer's input matrix. This product is usually the Frobenius inner product, and its activation function is commonly ReLU. As the convolution kernel slides along the input matrix for the layer, the convolution operation generates a feature map, which in turn contributes to the input of the next layer. This is followed by other layers such as pooling layers, fully connected layers, and normalization layers. Here it should be noted how close a convolutional neural network is to a matched filter.

7.

One of the simplest methods to prevent overfitting of a network is to simply stop the training before overfitting has had a chance to occur. It comes with the disadvantage that the learning process is halted.

8.

Drug discovery CNNs have been used in drug discovery. Predicting the interaction between molecules and biological proteins can identify potential treatments. In 2015, Atomwise introduced AtomNet, the first deep learning neural network for structure-based drug design. The system trains directly on 3-dimensional representations of chemical interactions. Similar to how image recognition networks learn to compose smaller, spatially proximate features into larger, complex structures, AtomNet discovers chemical features, such as aromaticity, sp3 carbons, and hydrogen bonding. Subsequently, AtomNet was used to predict novel candidate biomolecules for multiple disease targets, most notably treatments for the Ebola virus and multiple sclerosis.